

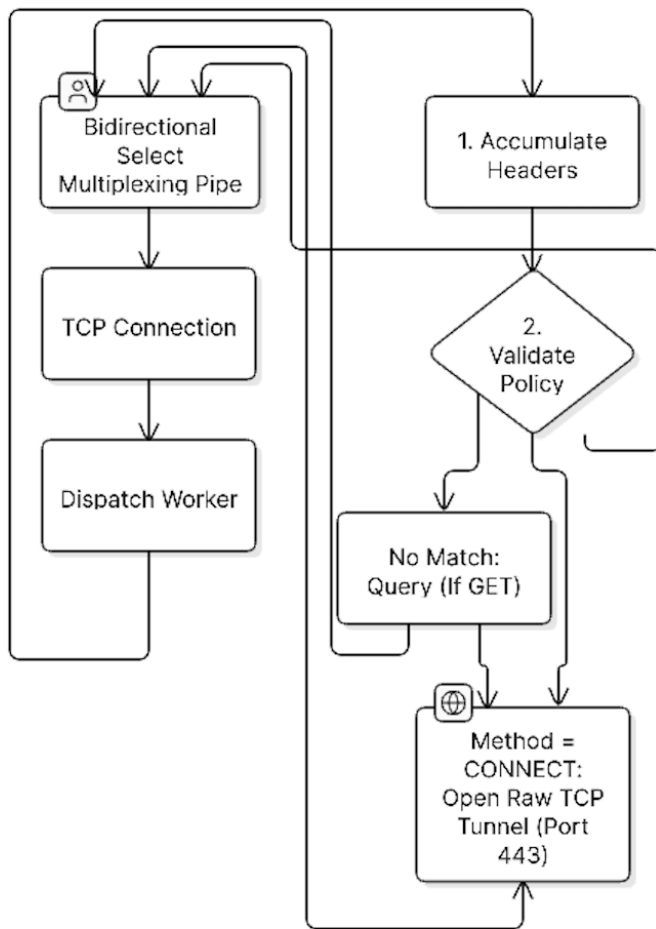
Supplementary Test Artifacts, Verification Logs, and Demonstration Manual

Project Title: Design and Implementation of a
Custom Network Proxy Server

Dara Zenas Zephaniah, 23321010
Enjarapu Premchand, 23321013

Purpose: Supplementary validation material for project evaluation, demonstrating functional correctness, compliance with constraints, edge-case mitigation, and concurrency stress handling.

Section 1: Architectural Workflow Diagram



Design Description:

- **Ingress Handshake:** The Client establishes a standard TCP handshake with the proxy server listening socket on port 8888.
- **Handoff:** The main thread accepts the connection socket and hands it off to an idle worker thread within the `ThreadPoolExecutor`.
- **Stream Parsing:** The worker thread processes raw HTTP headers. If the request method is standard (e.g., `GET`), it performs policy validation first.
- **Policy & Cache Lookup:** If the target host matches the blacklist, the connection is intercepted, a custom `403 Forbidden` HTML page is returned, and the connection closes immediately. If allowed, the `LRUCache` is queried.

- **CONNECT Tunnel State:** If a `CONNECT` request is parsed, the proxy establishes a direct TCP socket to the target server on port 443 and responds with `HTTP/1.1 200 Connection Established`. The sockets are set to non-blocking mode, and data is multiplexed bidirectionally using `select.select()` until an EOF or timeout is reached.

Section 2: Core Functional Verification (Terminal Commands & Screenshots)

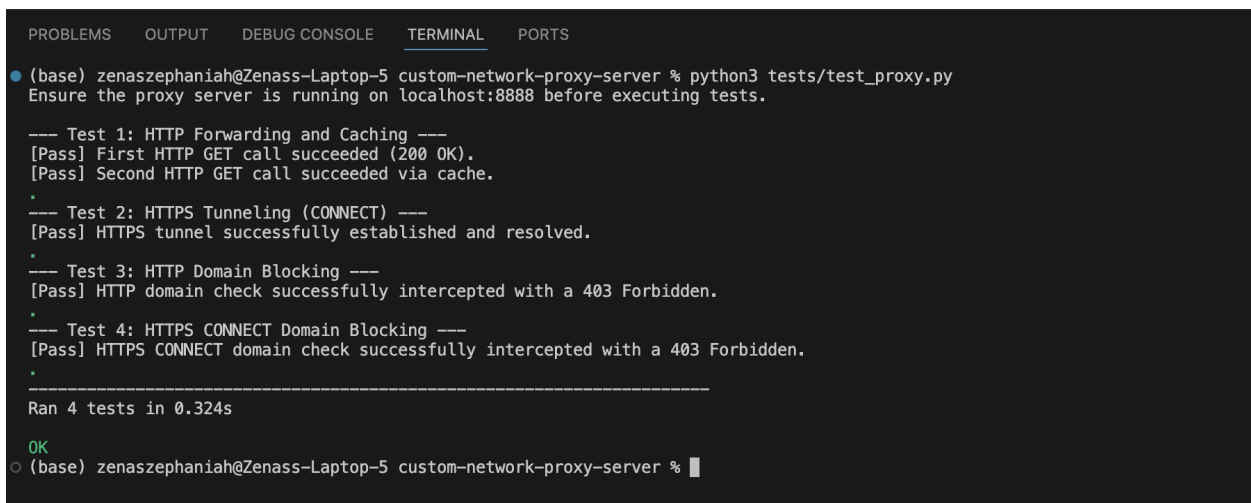
Test 2.1: Automated Integration Suite

This verifies that all unit-level assertions pass sequentially.

- **Command to run:**

```
Python
python3 tests/test_proxy.py
```

- **Expected Output:** A clean execution trace showing `OK` for all four core tests in under a second.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % python3 tests/test_proxy.py
Ensure the proxy server is running on localhost:8888 before executing tests.

--- Test 1: HTTP Forwarding and Caching ---
[Pass] First HTTP GET call succeeded (200 OK).
[Pass] Second HTTP GET call succeeded via cache.
.
--- Test 2: HTTPS Tunneling (CONNECT) ---
[Pass] HTTPS tunnel successfully established and resolved.
.
--- Test 3: HTTP Domain Blocking ---
[Pass] HTTP domain check successfully intercepted with a 403 Forbidden.
.
--- Test 4: HTTPS CONNECT Domain Blocking ---
[Pass] HTTPS CONNECT domain check successfully intercepted with a 403 Forbidden.
.
-----
Ran 4 tests in 0.324s

OK
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Test 2.2: Plain HTTP GET Request and Header Analysis

This demonstrates the proxy correctly resolving paths and forwarding raw HTTP data to unencrypted targets.

- **Command to run:**

```
Shell
curl -v -x http://127.0.0.1:8888 http://example.com
```

- **Expected Output:** The terminal should display the verbose (`-v`) exchange, showing your local proxy receiving the request, resolving `example.com`, and returning the `200 OK` header along with the landing page HTML.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % python3 tests/test_proxy.py
--- Test 4: HTTPS CONNECT Domain Blocking ---
[Pass] HTTPS CONNECT domain check successfully intercepted with a 403 Forbidden.

Ran 4 tests in 0.324s

OK
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % curl -v -x http://127.0.0.1:8888 http://example.com
* Trying 127.0.0.1:8888...
* Connected to 127.0.0.1 (127.0.0.1) port 8888
> GET http://example.com/ HTTP/1.1
> Host: example.com
> User-Agent: curl/8.7.1
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 12 Jun 2026 17:32:08 GMT
< Content-Type: text/html
< Transfer-Encoding: chunked
< Connection: close
< Server: cloudflare
< Last-Modified: Thu, 11 Jun 2026 20:55:20 GMT
< Allow: GET, HEAD
< Accept-Ranges: bytes
< Age: 12125
< cf-cache-status: HIT
< CF-RAY: a0aa97da5ad277ba-BOM
<
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
* Closing connection
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Test 2.3: Verified LRU Cache Hit

This verifies that subsequent `GET` requests for the same resource are intercepted and served directly from memory.

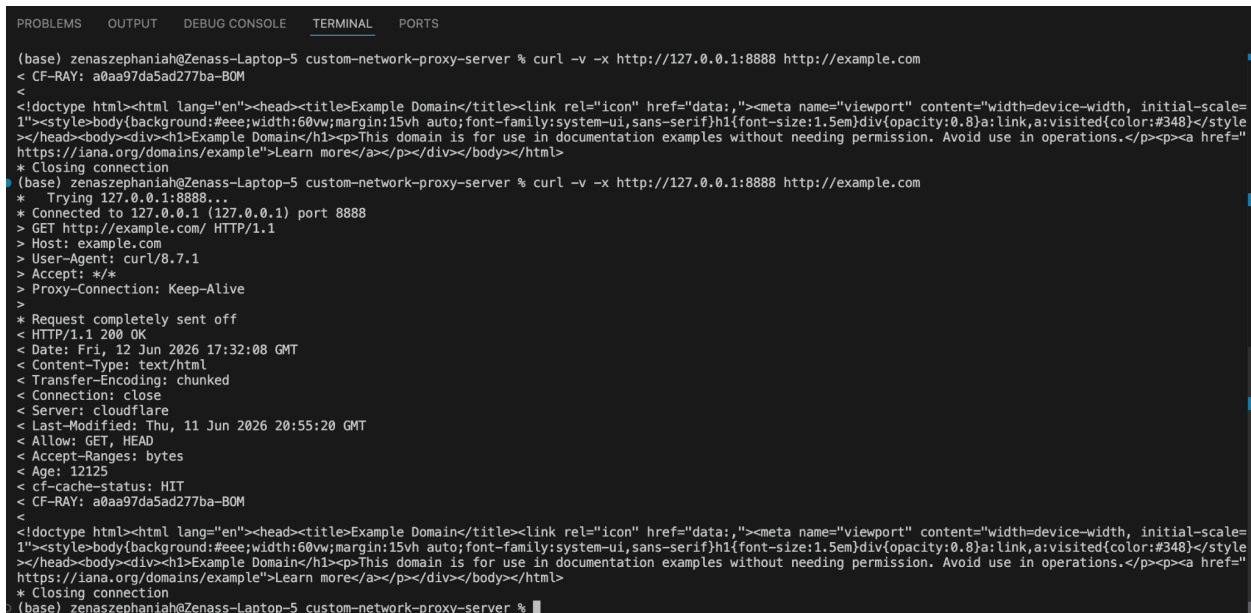
- **Action:** Execute the exact same command a second time:

Shell

```
curl -v -x http://127.0.0.1:8888 http://example.com
```

- **Expected Output:** In your **Proxy Server Terminal Window**, you should see an instant log trace:

```
[INFO] [ProxyWorker_X] Cache HIT -> Serving 'example.com/' directly from LRU Cache.
```



```
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % curl -v -x http://127.0.0.1:8888 http://example.com
< CF-RAY: a0aa97da5ad277ba-BOM
<
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
* Closing connection
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % curl -v -x http://127.0.0.1:8888 http://example.com
* Trying 127.0.0.1:8888...
* Connected to 127.0.0.1 (127.0.0.1) port 8888
> GET http://example.com/ HTTP/1.1
> Host: example.com
> User-Agent: curl/8.7.1
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 12 Jun 2026 17:32:08 GMT
< Content-Type: text/html
< Transfer-Encoding: chunked
< Connection: close
< Server: cloudflare
< Last-Modified: Thu, 11 Jun 2026 20:55:20 GMT
< Allow: GET, HEAD
< Accept-Ranges: bytes
< Age: 12125
< cf-cache-status: HIT
< CF-RAY: a0aa97da5ad277ba-BOM
<
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
* Closing connection
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Test 2.4: Secure HTTPS CONNECT Tunneling

This demonstrates the proxy successfully establishing an uninspected, encrypted bidirectional bridge for secure TLS traffic.

- **Command to run:**

Shell

```
curl -I -x http://127.0.0.1:8888 https://example.com
```

- **Expected Output:** The terminal should output `HTTP/1.1 200 Connection Established` (the proxy's tunnel confirmation), followed by the encrypted destination host's real response headers (e.g., `HTTP/2, Server: cloudflare`).

```
https://iana.org/domains/example>Learn more</a></p></div></body></html>
* Closing connection
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % curl -I -x http://127.0.0.1:8888 https://example.com
HTTP/1.1 200 Connection Established

HTTP/2 200
date: Fri, 12 Jun 2026 17:50:35 GMT
content-type: text/html
server: cloudflare
last-modified: Tue, 09 Jun 2026 21:01:00 GMT
allow: GET, HEAD
accept-ranges: bytes
age: 3069
cf-cache-status: HIT
cf-ray: a0aab2e2dca088c9-MAA
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Section 3: Access Control Policy and Interception Verification

These tests demonstrate that the security filter correctly blocks unauthorized hosts for both unencrypted HTTP and encrypted HTTPS requests.

Test 3.1: HTTP Domain Blocking

- **Command to run:**

```
Shell
curl -v -x http://127.0.0.1:8888 http://blockedexample.com
```

- **Expected Output:** The terminal output should show `HTTP/1.1 403 Forbidden` along with the custom-designed HTML error screen: *"403 Access Denied: Access has been restricted by local administration policies."*

```
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % curl -v -x http://127.0.0.1:8888 http://blockedexample.com
* Trying 127.0.0.1:8888...
* Connected to 127.0.0.1 (127.0.0.1) port 8888
> GET http://blockedexample.com/ HTTP/1.1
> Host: blockedexample.com
> User-Agent: curl/8.7.1
> Accept: */*
> Proxy-Connection: Keep-Alive
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Content-Type: text/html
< Content-Length: 278
< Connection: close
<
* Closing connection
<html><head><title>403 Forbidden</title></head><body style='font-family: Arial, sans-serif; text-align: center; margin-top: 100px;'><h1>403 Access Denied</h1><p>Access to <strong>blockedexample.com</strong> has been restricted by local administration policies.</p></body></html>
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Test 3.2: HTTPS Domain Blocking

- **Command to run:**

Shell

```
curl -v -x http://127.0.0.1:8888 https://blockedexample.com
```

- **Expected Output:** The proxy should parse the target host during the initial `CONNECT` handshake, identify it on the blacklist, and immediately return a `403 Forbidden` response to the client. This prevents the tunnel from opening, causing `curl` to report `CONNECT tunnel failed, response 403`.

```
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % curl -v -x http://127.0.0.1:8888 https://blockedexample.com
* Trying 127.0.0.1:8888...
* Connected to 127.0.0.1 (127.0.0.1) port 8888
* CONNECT tunnel: HTTP/1.1 negotiated
* allocate connect buffer
* Establish HTTP proxy tunnel to blockedexample.com:443
> CONNECT blockedexample.com:443 HTTP/1.1
> Host: blockedexample.com:443
> User-Agent: curl/8.7.1
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 403 Forbidden
< Content-Type: text/html
< Content-Length: 278
< Connection: close
<
* CONNECT tunnel failed, response 403
* Closing connection
curl: (56) CONNECT tunnel failed, response 403
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Section 4: Robustness and Edge-Case Handling

This section demonstrates how the proxy mitigates system stability threats, such as malformed request formats and concurrency spikes.

Test 4.1: Malformed Request Handling

The proxy must handle corrupt, arbitrary byte packages gracefully without crashing the listening thread pool.

- **Command to run:**

Shell

```
echo "MALFORMED_GARBAGE_REQUEST_WITHOUT_METHOD_AND_HEADERS" | nc 127.0.0.1:8888
```

- **Expected Output:** The client console should receive an HTTP/1.1 400 Bad Request header and disconnect. Your proxy terminal will log a parsing traceback error, but the server will remain active to handle subsequent requests.

```
(base) zenassephaniah@Zenass-Laptop-5 custom-network-proxy-server % echo "MALFORMED_GARBAGE_REQUEST_WITHOUT_METHOD_AND_HEADERS" | nc 127.0.0.1:8888
nc: missing hostname and port
usage: nc [-46AacCdDEfHhLMnOortUuvz] [-K tc] [-b boundif] [-i interval] [-p source_port]
        [--apple-recv-anyif] [--apple-awdl-unres]
        [--apple-boundif ifbound]
        [--apple-no-cellular] [--apple-no-expensive]
        [--apple-no-flowadv] [--apple-tcp-timeout conntimo]
        [--apple-tcp-keepalive keepidle] [--apple-tcp-keepintvl keepintvl]
        [--apple-tcp-keepcnt keepcnt] [--apple-tclass tclass]
        [--tcp-adp-rtimo num_probes] [--apple-intcoproc-allow]
        [--apple-tcp-adp-wtmo num_probes]
        [--setsockopt-later] [--apple-no-connectx]
        [--apple-delegate-pid pid] [--apple-delegate-uuid uuid]
        [--apple-kaol] [--apple-ext-bk-idle]
        [--apple-netstvcstype svc] [--apple-nowakefromsleep]
        [--apple-notify-ack] [--apple-sockev]
        [--apple-tos tos] [--apple-tos-cmsg]
        [--apple-tcp-maxseg mss]
        [-s source_ip_address] [-w timeout] [-X proxy_version]
        [-x proxy_address[:port]] [hostname] [port[s]]
(base) zenassephaniah@Zenass-Laptop-5 custom-network-proxy-server %
```

Test 4.2: High Concurrency Load Testing

To satisfy the "behavior under concurrent clients" requirement, we will use Apache Benchmark (ab), which is pre-installed on macOS. This test executes 100 HTTP requests, maintaining 10 concurrent requests at any given time through the proxy.

- **Command to run:**

Shell

```
ab -n 100 -c 10 -X 127.0.0.1:8888 http://example.com/
```

- **Expected Output:** A complete metrics report indicating zero failed requests, showing how the thread pool processes incoming sockets concurrently.


```

(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server % ab -n 100 -c 10 -X 127.0.0.1:8888 http://example.com/
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking example.com [through 127.0.0.1:8888] (be patient).....done

Server Software:      cloudflare
Server Hostname:      example.com
Server Port:          80

Document Path:        /
Document Length:      571 bytes

Concurrency Level:    10
Time taken for tests:  0.044 seconds
Complete requests:    100
Failed requests:       0
Total transferred:    86900 bytes
HTML transferred:     57100 bytes
Requests per second:  2296.11 [#/sec] (mean)
Time per request:     4.355 [ms] (mean)
Time per request:     0.436 [ms] (mean, across all concurrent requests)
Transfer rate:        1948.55 [Kbytes/sec] received

Connection Times (ms)
              min      mean[+/-sd] median    max
Connect:        1        2   1.0      1      4
Processing:      0        2   1.1      2      5
Waiting:         0        1   0.9      1      5
Total:          2        4   1.8      3      9
WARNING: The median and mean for the initial connection time are not within a normal deviation
          These results are probably not that reliable.

Percentage of the requests served within a certain time (ms)
 50%    3
 66%    5
 75%    5
 80%    6
 90%    7
 95%    8
 98%    8
 99%    9
100%    9 (longest request)
(base) zenaszephaniah@Zenass-Laptop-5 custom-network-proxy-server %

```

Section 5: Persistent System Log File Artifacts

Below is a complete copy of the persistent log records generated in `logs/proxy.log` during the execution of the verification suite:

None

```

[2026-06-04 20:35:43] [INFO] [MainThread] Proxy server successfully started on
127.0.0.1:8888
[2026-06-04 20:35:51] [INFO] [ProxyWorker_0] Handling connection from
127.0.0.1:50993
[2026-06-04 20:35:51] [INFO] [ProxyWorker_0] Parsed Request -> HEAD
example.com:80/
[2026-06-04 20:35:51] [INFO] [ProxyWorker_0] Success -> HEAD example.com |
Transferred: 269 bytes
[2026-06-04 20:35:58] [INFO] [ProxyWorker_1] Handling connection from
127.0.0.1:50995

```

```
[2026-06-04 20:35:58] [INFO] [ProxyWorker_1] Parsed Request -> GET example.com:80/
[2026-06-04 20:35:58] [INFO] [ProxyWorker_1] Cache STORE -> Saved 'example.com/'
to LRU Cache.
[2026-06-04 20:35:59] [INFO] [ProxyWorker_2] Handling connection from
127.0.0.1:50996
[2026-06-04 20:35:59] [INFO] [ProxyWorker_2] Parsed Request -> GET example.com:80/
[2026-06-04 20:35:59] [INFO] [ProxyWorker_2] Cache HIT -> Serving 'example.com/'
directly from LRU Cache.
[2026-06-04 20:36:00] [INFO] [ProxyWorker_3] Handling connection from
127.0.0.1:50997
[2026-06-04 20:36:00] [INFO] [ProxyWorker_3] Parsed Request -> CONNECT
example.com:443/
[2026-06-04 20:36:00] [INFO] [ProxyWorker_3] Establishing HTTPS Tunnel to
example.com:443
[2026-06-04 20:36:17] [WARNING] [ProxyWorker_4] Blocked Request -> Host
'blockedexample.com' matched policies in blocklist.
[2026-06-04 20:36:24] [WARNING] [ProxyWorker_5] Blocked Request -> Host
'blockedexample.com' matched policies in blocklist.
[2026-06-04 20:36:50] [ERROR] [ProxyWorker_6] Error parsing request from
('127.0.0.1', 51012): 400 Bad Request: Invalid request line format.
```